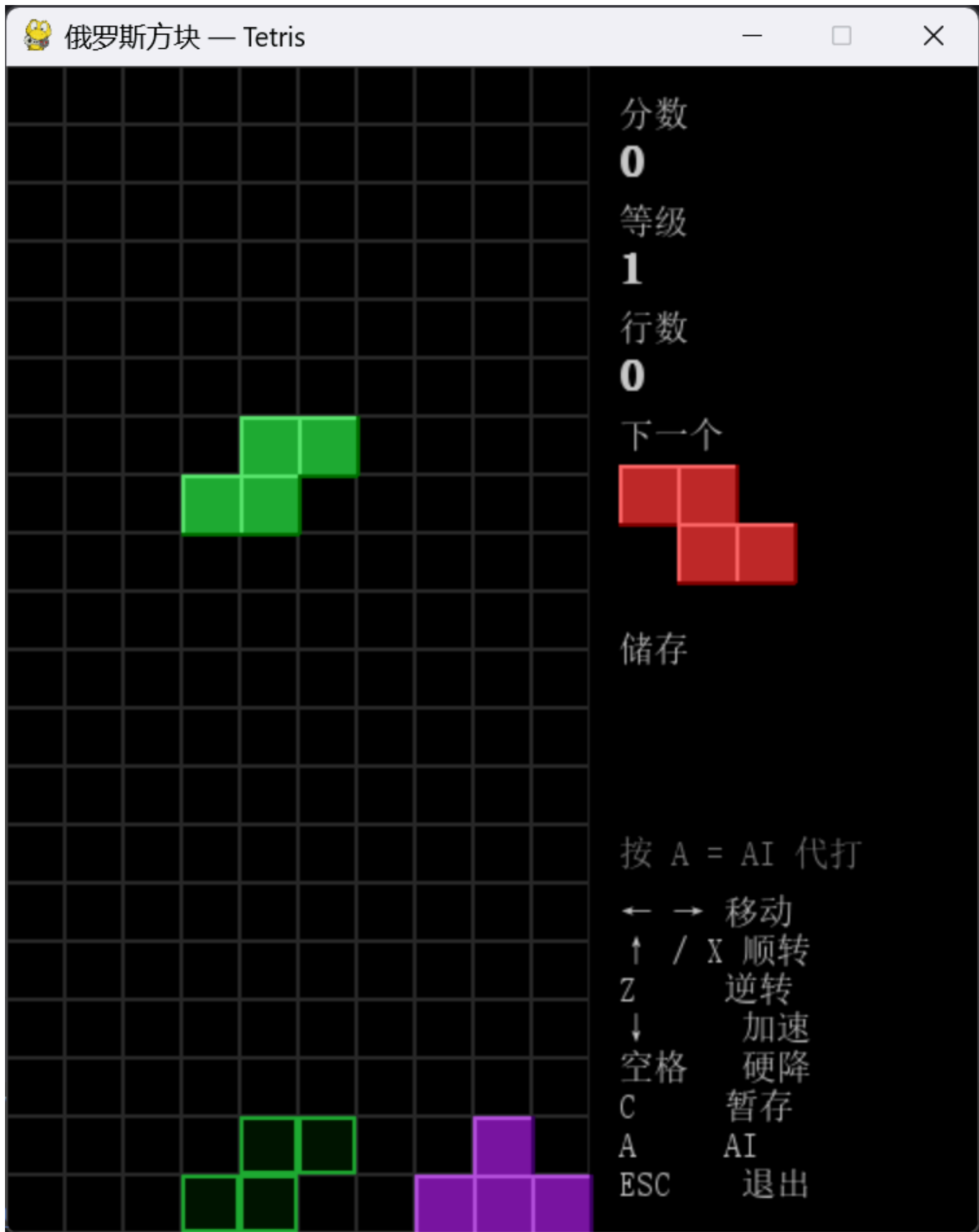

一、项目 GitHub 仓库链接

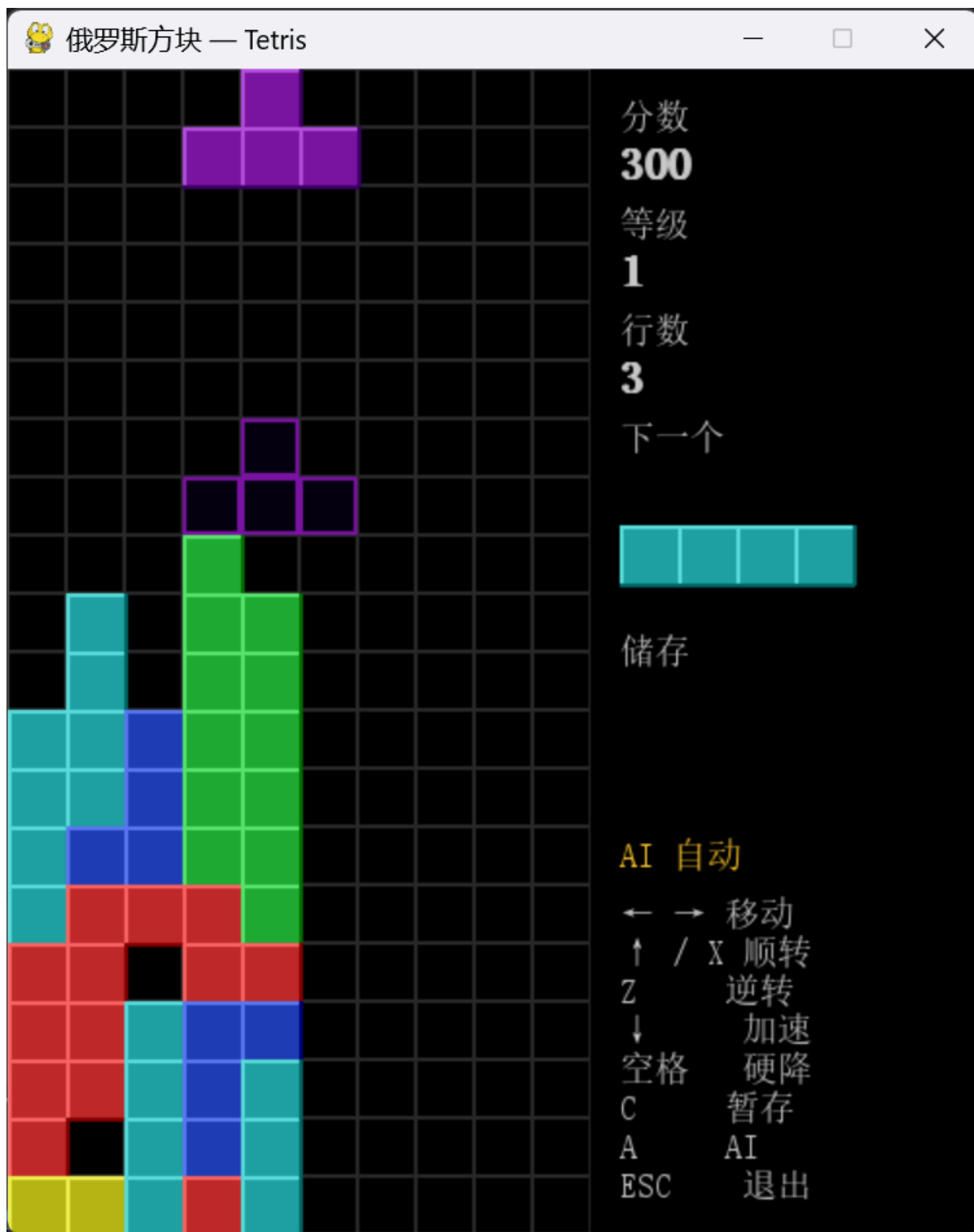
<https://github.com/taiy5/Tetris>

二、项目核心截图

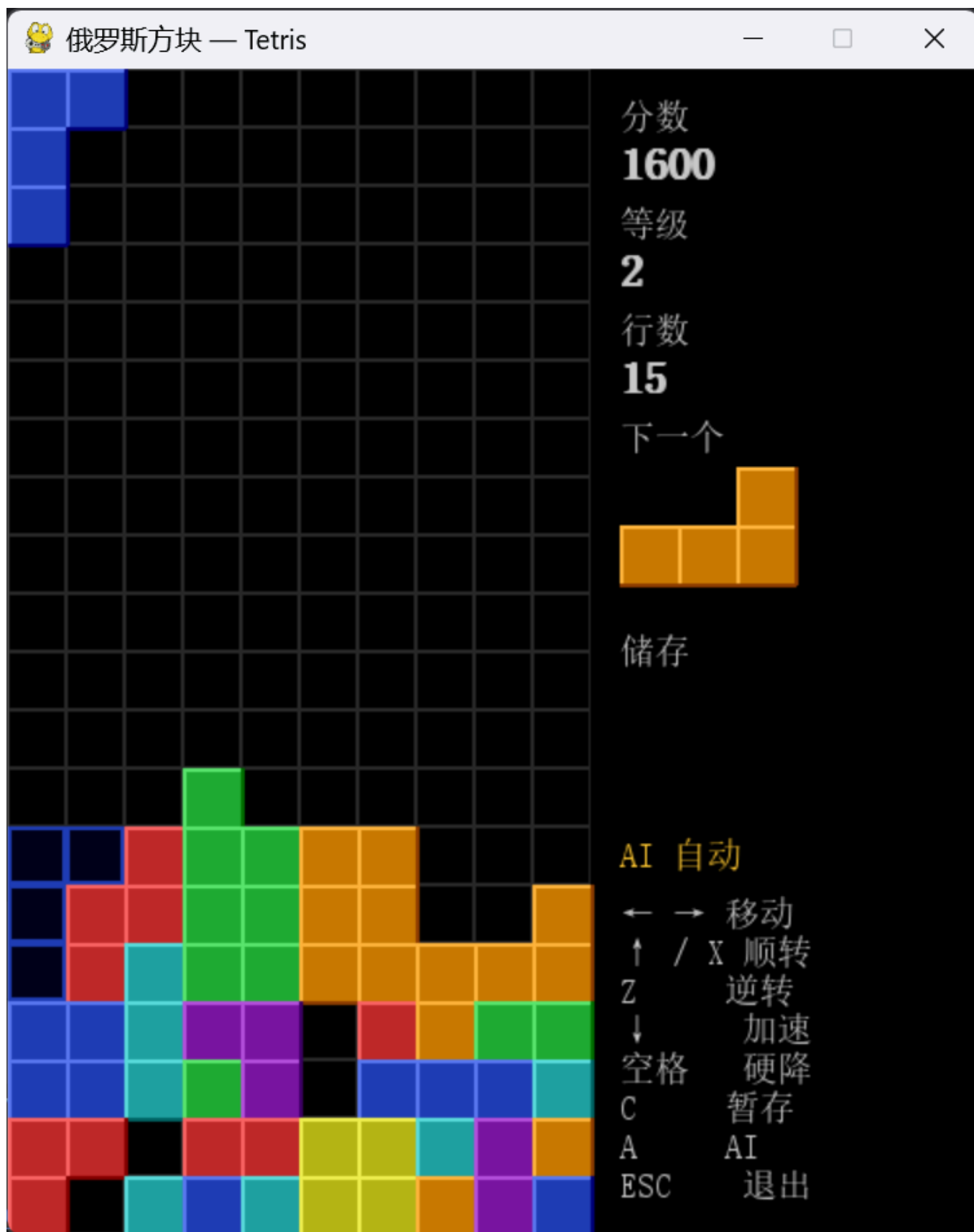
游戏主界面



AI 模式运行中



消行效果



三、README 内容预览

CONTRIBUTORS	1
FORKS	0
STARS	1
ISSUES	0 OPEN
LICENSE	MIT

俄罗斯方块 — Tetris

一个使用 Python + Pygame 编写的经典俄罗斯方块游戏
含 AI 自动游玩模式，基于 Dellacherie 启发式评价函数

[快速开始](#) · [操作说明](#) · [AI 自动游玩](#) · [功能特性](#)

► 目录

关于项目

项目背景

俄罗斯方块是电子游戏史上最经典的益智游戏之一。本项目的选题初衷是：

- **从零实现完整的游戏逻辑**：不同于调用现成的游戏引擎，本项目从碰撞检测、方块旋转、消行计分、等级加速到游戏结束判定，全部以二维数组操作的方式手工实现，深入理解游戏底层的状态管理机制。
- **探索 AI 如何玩人类游戏**：俄罗斯方块的决策空间虽然不大（每方块约 40 种落点），但在高速下落时需要瞬间做出最优选择。本项目实现了基于启发式评价函数的 AI，将"方块掉在哪里最好"转化为一个最优化搜索问题。

技术栈

- **Python 3.x**
- **Pygame** — 跨平台游戏开发库
- **Dellacherie Algorithm** — Tetris AI 经典评价函数（权重经 PSO 优化）

[\(回到顶部\)](#)

功能特性

- **7 种经典方块** — I、O、T、L、J、S、Z，完整 4 旋转状态机
- **双方向旋转** — Up/X 顺时针，Z 逆时针（带墙踢偏移）
- **墙踢 (Wall Kick)** — 贴墙旋转自动偏移 $\pm 1 \sim 2$ 格，不会卡死

- **虚影预览（Ghost Piece）** — 半透明落点提示，精准放置
- **暂存系统（Hold）** — 按 C 键暂存 / 交换方块，每块限用一次
- **硬降（Hard Drop）** — 空格键直接落底锁定
- **DAS 连续移动** — 按住方向键先等 170ms，之后每 50ms 连续移动
- **等级加速** — 每消 10 行升一级，下落速度 500ms 到 50ms 递减
- **经典计分** — 单消 100 / 双消 300 / 三消 500 / 俄罗斯方块 800
- **长按重开** — 游戏进行中长按 R 1 秒重新开局
- **AI 代打** — 按 A 切换，Dellacherie 启发式搜索，自动不停重开

[\(回到顶部\)](#)

快速开始

环境要求

- Python 3.7 或更高版本
- pip（Python 包管理器）

安装与运行

1. 克隆仓库

```
git clone https://github.com/taiy5/Tetris.git
cd Tetris
```

2. 安装 Pygame 依赖

```
pip install -r requirements.txt
```

3. 运行游戏

```
python src/tetris.py
```

[\(回到顶部\)](#)

操作说明

按键	功能
Left/Right	左 / 右移动方块
Down	加速下落（软降）
Up 或 X	顺时针旋转 90 度
Z	逆时针旋转 90 度

按键	功能
空格	硬降（直接落底并锁定）
C	暂存当前方块 / 取出暂存方块
A	切换 AI 自动游玩模式
R (长按 1s)	游戏中重新开始
R (点按)	游戏结束后重新开始
ESC	退出游戏

[\(回到顶部\)](#)

游戏规则

基本规则

- 方块从 **10x20 网格** 顶部持续下落
- 玩家可左右移动、旋转方块，加速下落或一次性落底
- 当一行被方块**完全填满**时，该行消除，上方所有方块下移一行
- 方块堆积触及顶部时**游戏结束**

计分系统

同时消除行数	得分	名称
1 行	100	单消 (Single)
2 行	300	双消 (Double)
3 行	500	三消 (Triple)
4 行	800	俄罗斯方块 (Tetris)

等级系统

- 初始等级 **1**，每消除 **10 行** 升一级
- 下落速度随等级递增：500ms -> (逐级 -40ms) -> 最低 50ms

[\(回到顶部\)](#)

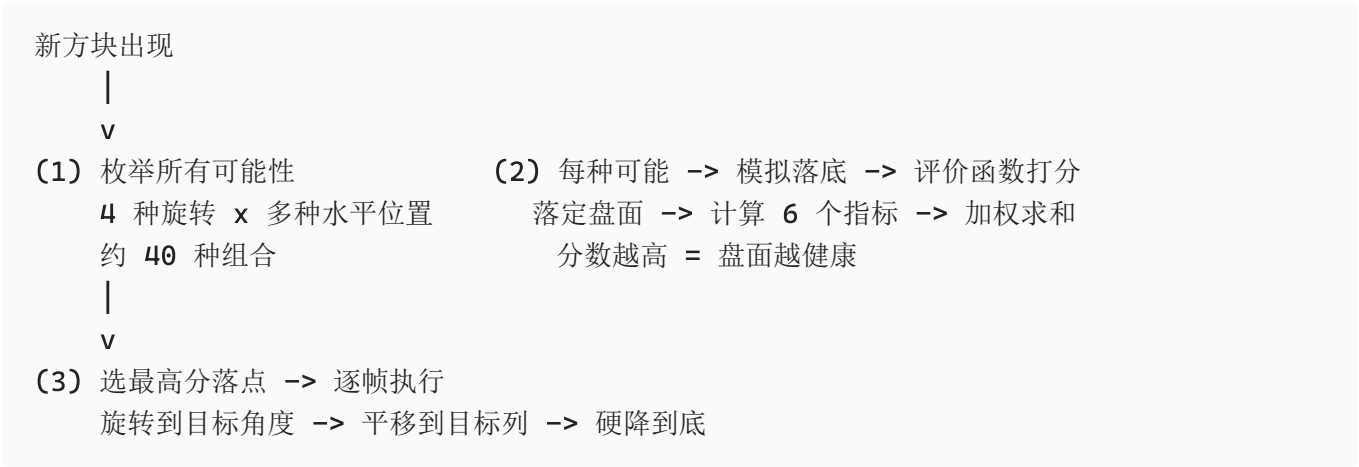
AI 自动游玩

如何启动

运行游戏后按 **A** 键即进入 AI 模式，右侧面板显示黄色 **"AI 自动"**。再按 A 切回手动。AI 模式下游戏结束后自动等待 0.8 秒重开。

AI 工作原理

每当新方块出现，AI 做三件事：



评价函数六维度

AI 使用了 **Dellacherie Algorithm** 的经典评价函数，权重经粒子群优化（PSO）调参。每个落点模拟落底后计算以下 6 个特征并加权求和：

维度	含义	权重	方向
聚合高度	各列方块堆叠高度的总和	-45	越低越好
消行奖励	消 1 行 +3400 / 2 行 +12000 / 3 行 +20000 / 4 行 +34000	—	越多越好
行内跳变	行内"有块-空"的切换次数	-32	越少越整齐
列内跳变	列内"有块-空"的切换次数	-93	越少越整齐
空洞	被方块盖住的空位数量	-79	越少越好
井深	某列明显低于两侧邻居的深度	-34	越浅越好

权重来源：Colin Fahey (2007) 用 PSO 在 Dellacherie 框架上调参得到的最佳组合，是 Tetris AI 社区的公认标准。

AI 速度调节

AI 每步操作之间有 **80ms** 冷却（`_ai_step_delay >= 80`），修改这个值即可调速：

- 数值越小 -> AI 越快
- 数值越大 -> AI 越慢，方便观察每一步决策

项目结构

```
Tetris/
├─ src/
│   └─ tetris.py      <- 游戏主程序（单文件，含全部游戏 + AI 代码）
├─ docs/              <- 文档与演示截图
├─ tests/             <- 测试目录
├─ README.md          <- 项目说明文档
├─ requirements.txt   <- 环境依赖清单
└─ LICENSE             <- MIT 开源协议
```

代码架构

常量区：棋盘尺寸 / 颜色 / 7 种方块 x 4 旋转状态数据

```
|
v
Tetris 类
├─ __init__()          初始化 Pygame，创建窗口和字体
├─ reset()             重置全部游戏状态
├─ spawn_piece()       方块生成（next -> current）
├─ get_current_blocks() 获取当前方块 4 格局部坐标
├─ collides()          碰撞检测 - 所有操作的前置判断
├─ lock_piece()        锁定方块 -> 消行 -> 发新牌
├─ clear_lines()       扫描满行 -> 删除 -> 计分 -> 更新等级
├─ move()              方块位移
├─ rotate()            旋转 + 墙踢偏移（支持正反方向）
├─ hold_piece()        暂存 / 交换方块（hold_used 限用一次）
├─ drop_hard()         硬降到底 -> 锁定
├─ get_ghost_y()       计算虚影落点 Y 坐标
├─
├─ _ai_simulate_drop() AI 模拟引擎：深拷贝棋盘 -> 落底 -> 消行
├─ _ai_evaluate()      AI 评价函数：6 维度加权打分（核心）
├─ _ai_find_best()     AI 决策入口：枚举旋转 x 位置，择最高分
├─
├─ handle_input()      键盘轮询 + DAS 三阶段状态机 + ESC/AI 切换
├─ update()            自动下落 / AI 规划与逐帧执行
├─ draw_cell()         画一个带立体感的格子
├─ draw()              渲染整帧（6 层绘制顺序 + 右侧面板）
└─ run()               游戏主循环（60 FPS）
```

核心算法与数据结构

本节对照课程知识点，逐项说明本项目中"用了什么"以及"用在哪里"。

1. 二维数组与矩阵操作

对应知识点：数组、矩阵

整个游戏建立在 `grid = [ROWS][COLS]` 的二维数组之上。方块锁定（写入颜色值）、消行（删除整行后 `insert(0, ...)` 顶部补空行）、碰撞检测（遍历 4 个格子在棋盘上的投影坐标）均为对二维数组的增删改查。所有操作的时间复杂度均为 $O(n)$ ，其中 n 为棋盘格子总数 200。

2. 墙踢（Wall Kick）与试探法

对应知识点：贪心试探、边界条件处理

旋转后如果新状态碰撞了，代码不直接放弃，而是依次试探 ± 1 、 ± 2 的水平偏移：

```
for offset in (1, -1, 2, -2):
    if not collides(new_blocks, x + offset, y):
        x += offset; rotation = new_rotation; return
```

这是一种局部贪心策略——贪心地收下第一个能找到的合法偏移。每次旋转最多 5 次碰撞检测，单次 $O(4)$ 。

3. 碰撞检测 — 全遍历判定

对应知识点：边界条件判断、遍历

`collides()` 是游戏中使用频率最高的函数（每次移动 / 旋转 / 硬降 / AI 模拟均调用了它）。它对每个格子的 (gx, gy) 做三个判断：

- ① `gx < 0 or gx >= COLS` → 撞左/右墙
- ② `gy >= ROWS` → 超出底部
- ③ `grid[gy][gx] != BLACK` → 与已锁定方块重叠

`gy < 0` 的情况被**有意跳过**——允许方块从屏幕上方伸出去一部分，这保证了方块出生时即使局部坐标 $y=0$ 的格子还没完全进入棋盘也不会触发误判。

4. 消行 — 倒序删除模式

对应知识点：数组删除的索引偏移问题

```
for r in sorted(full_rows, reverse=True):    # 从底部往顶部删
    del grid[r]                             # 索引不会被还没删的高行影响
```

如果正序删除（从顶部往底部），删了第 3 行后，原本的第 5 行会变成第 4 行，后续的删除目标行号就错了。倒序删除是数组结构下处理"边遍历边删"的标准手法。

5. AI 启发式搜索 — 枚举 + 评价函数

对应知识点：状态空间搜索、启发式评价函数、剪枝

这是本项目与课程知识点关联最深的部分：

步骤	算法	复杂度
枚举所有落点	遍历：4 个旋转 x ~10 个水平位置	O(40)
模拟落底	贪心：逐步下移直到碰撞	O(20)
锁定 + 消行	遍历 4 格 + 扫描 20 行	O(20+4)
评价函数打分	6 个特征的计算均独立遍历棋盘	O(6 x 200)

总计每个方块最多 40 x (20 + 24 + 1200) 次操作，在 60 FPS 下一次 update 调用内完成。评价函数的 6 个维度（聚合高度、消行奖励、行跳变、列跳变、空洞、井深）本质上是**人为设计的启发式特征**——AI 不保证全局最优，但保证在 O(n) 时间内给出一个经过验证的近似最优解。更精确的全局最优搜索（如 minimax 搜索未来多步）会因指数爆炸而不可行，因此启发式近似是本场场景下的正确取舍。

[\(回到顶部\)](#)

AI 工具声明

本项目使用了 **Claude (Anthropic)** 作为编程辅助工具，具体分工如下：

部分	AI 辅助程度	说明
游戏 GUI 框架	高	Pygame 窗口创建、绘制循环、字体渲染等样板代码由 AI 生成
输入系统（DAS / 轮询）	高	键盘轮询 + DAS 三阶段状态机由 AI 协助实现及调试
README 文档撰写	中	结构框架由 AI 辅助完成，内容由人工填写
代码注释	中	AI 添加可阅读的详细注释

部分	AI 辅助程度	说明
AI 评价函数（Dellacherie）	自主	查询文献得到六维度权重公式，人工完成实际的计算实现
方块旋转、方块定义数据、碰撞检测、消行、计分逻辑	自主	全部由人工学习经典代码后实现
功能设计	自主	功能根据作者实际游玩经历自主复现

所有 AI 生成的代码均经过了人工阅读、理解和验证。

([回到顶部](#))

许可证

本项目基于 MIT License 开源。

([回到顶部](#))